

Welcome!

Ok, so you want to learn how to write Bash scripts on Unix/Linux. Or, it's part of a subject you're doing and so you're learning because you have to. Either way, that's great. Keep on reading my friend as you are about to harness a powerful tool that will allow you to perform complex repetitive tasks with minimal effort.



SECTION BREAKDOWN

[Introduction](#)

[Outline](#)

[Structure](#)

[General House Rules](#)

[Problem Solving and Creativity](#)

[About the Author](#)

[Using this Material](#)

[Disclaimer](#)

NEXT SECTION

[What is a Bash Script?](#)

Introduction

The following pages are intended to give you a solid foundation in how to write Bash scripts, to get the computer to do complex, repetitive tasks for you. You won't be a bash guru at the end but you will be well on your way and armed with the right knowledge and skills to get you there if that's what you want (which you should).

Bash scripts are used by Systems Administrators, Programmers, Network Engineers, Scientists and just about anyone else who uses a Linux/ Unix system regularly. No matter what you do or what your general level of computer proficiency is, you can generally find a way to use Bash scripting to make your life easier. Bash is a command line language. The name stands for **Bourne Again SHell**. It is an open source version of the Bourne Shell and was first released in 1989.

BASH is the default shell on most Linux distributions and Apple's macOS (formerly OS X). Recently, a version has also been made available for Windows 10.

Outline

This Bash scripting tutorial is divided into 8 sections. In general I recommend you work through them in order but if you've come here just to learn about a specific topic then who am I to slow you down, just head straight on over.

You can now jump into section 1 and get started or keep reading below to learn a little more about this tutorial.

1. [What is a Bash Script](#) - What are they, how do they work and how to run them.
2. [Variables](#) - Store data temporarily for later use.
3. [Input](#) - Different ways to supply data and directions to your Bash script.
4. [Arithmetic](#) - Perform various arithmetic operations in your Bash script.
5. [If Statements](#) - How to make decisions within your Bash script.
6. [Loops](#) - A variety of ways to perform repetitive tasks.
7. [Functions](#) - Reuse code to make life easier.
8. [User Interface](#) - Make your scripts user friendly.

Tip

Bash scripting relies on an understanding of the Linux Command Line. If you are not confident on the Linux Command Line I suggest you start by going through our [Linux Tutorial](#).

Structure

The best way to learn Bash Scripting is as a series of small, easy to manage steps. This tutorial is organised as such, with each section building upon the knowledge and skills learned in the previous sections. If you work through them in order, read them fully (there is a fair bit of material but it is important for getting a proper understanding) and practice on the command line as you go I believe you should have a fairly pleasant and smooth journey to Bash scripting mastery.

99 times out of 100 when a student has troubles with this stuff I find that it is not that they are incapable but that they were lazy and didn't read the material fully.

Each section is structured in the following format:

- An introduction outlining what you will learn in that section.
- Detailed material including extensive examples.
- A summary of the most important concepts.
- A set of activities to help you solidify your knowledge and skills.

Think of the activities not as tutorial questions (such as you may get in a class at school) but as direction on where to explore in the Linux environment to benefit from your new knowledge and skills. Treat the activities as a starting point for exploration. The further you take it, the better you will do. (How well you do and how far you go depends on how unlazy you are.)

This site is also designed to work well on tablets. I know a lot of students like to be at their computer doing work and have their tablet next to them with reference material on it. If that is you then this site works quite well with that set up. Another good approach is to have your browser on one half of the screen of your computer and a terminal on the other half so you can try out the examples as you go.

General House Rules

- I'll refer to Linux in the following pages, whenever I do, assume I'm actually saying Unix/ Linux. Linux is an offshoot of Unix and behaves pretty much exactly the same.
- Whenever you see **<something>**, what this means is that you are to replace this with something useful. Replace the whole thing (including the < and >). If you see something such as **<n>** then it usually means replace this with a number.
- Whenever you see **[something]** this usually means that this something is optional. When you run the command you may put in the something or leave it out.

Problem Solving and Creative Thinking

If you wish to succeed with Bash scripting then there are two things you need, Problem solving and Creative thinking. Here are some basic pointers to help you along the way.

- **Explore and experiment.** Remember, you're learning how to assemble a set of building blocks and with them you can build almost anything. The examples you will find are intended to be an illustration of how they work, not the only thing you can do with them. I encourage you to tweak the examples and see how they behave. This will give you a much better understanding on how they work. You will have a lot of questions along the way along the lines of "What if....?" and "Can I ...?" to which I say, "Give it a go and see what happens." The worst you can really get is an error message. In which case you read the error message to understand why it didn't work, then have another go. Don't hold back!
- **Read carefully** and don't skip over the fine details. I can't stress this enough. The fine details are important and are often the difference between your script working and not working. If something isn't working then re-read the material carefully and re look over what you have typed in to make sure you haven't make a silly little typo.

To learn more about problem solving visit our [Problem Solving Skills](#) tutorial.

About the Author



Hi. My name is Ryan Chadwick and I have been teaching Linux to students for over 10 years now. It's something that I very much enjoy. In a world where we are increasingly hiding the complexities away from users (smart phones and tablets in particular), it is always satisfying to give that power back to people and allow them to do much more with their technology.

But teaching is only one of the things I do. I also develop and manage websites, and manage computer systems for a few different organisations. I feel that all my work is complementary and benefits from what I learn in the others. Combined, they give me a fairly good understanding of how technology works, and also an understanding of how people use technology, perceive technology, learn technology and work around technology.

This website is both for you, and for me. For you, it is a digital form of what I have been teaching my students over the years. For me, it is an opportunity to clarify and polish what I teach and how I deliver it. To these ends, if you have any feedback at all, be it to tell me you found a particular section confusing, or to let me know it was helpful, I would be more than happy to hear from you.

Drop us a message at ryantutorial@todolistme.net

Using this material

Anyone may link to the content on this website.

If you are a teacher, academic, manager or trainer and wish to use this material in your classes or training you are more than welcome to link to these pages and refer your students here or show them in your classes, but please do not just copy my material.

I've put a fair amount of time and effort into creating this resource so please be cool and do the right thing.

Disclaimer

Unfortunately, in this crazy and litigious world we now live in, one has to keep themselves covered so here is my disclaimer.

To the best of my knowledge, the material contained within these pages is correct. I make no guarantees however. I also take no responsibility for any damage you may do based upon knowledge gained from here (even though the chances of doing so are relatively small at best). Please use common sense when using any of the knowledge and skills gained from using this material.

[What is a Bash Script](#) ➤

What is a Bash Script?

Let's Dive In



SECTION BREAKDOWN

[Introduction](#)

[So what are they exactly?](#)

[How do they work?](#)

[How do we run them?](#)

[Formatting](#)

[NEXT SECTION](#)

[Variables](#)

Introduction

This page is mostly foundation information. It's kinda boring but essential stuff that will help you to appreciate why and how certain things behave the way they do once we start playing about with the fun stuff (which I promise we'll do in the next section). Taking the time to read and understand the material in this section will make the other sections easier to digest so persevere and it'll be well worth your time.

So what are they exactly?

Think of a script for a play, or a movie, or a TV show. The script tells the actors what they should say and do. A script for a computer tells the computer what it should do or say. In the context of Bash scripts we are telling the Bash shell what it should do.

A Bash script is a plain text file which contains a series of commands. These commands are a mixture of commands we would normally type ourselves on the command line (such as `ls` or `cp` for example) and commands we could type on the command line but generally wouldn't (you'll discover these over the next few pages). An important point to remember though is:

Anything you can run normally on the command line can be put into a script and it will do exactly the same thing. Similarly, anything you can put into a script can also be run normally on the command line and it will do exactly the same thing.

You don't need to change anything. Just type the commands as you would normally and they will behave as they would normally. It's just that instead of typing them at the command line we are now entering them into a plain text file. In this sense, if you know how to do stuff at the command line then you already know a fair bit in terms of Bash scripting.

It is convention to give files that are Bash scripts an extension of `.sh` (myscript.sh for example). As you would be aware (and if you're not maybe you should consider reviewing our [Linux Tutorial](#)), Linux is an extensionless system so a script doesn't necessarily have to have this characteristic in order to work.

How do they work?

This is just a little bit of background knowledge. It's not necessary to understand this in order to write scripts but it can be useful to know once you start getting into more complex scripts (and scripts that call and rely on other scripts once you start getting really fancy).

In the realm of Linux (and computers in general) we have the concept of programs and processes. A program is a blob of binary data consisting of a series of instructions for the CPU and possibly other resources (images, sound files and such) organised into a package and typically stored on your hard disk. When we say we are running a program we are not really running the program but a copy of it which is called a process. What we do is copy those instructions and resources from the hard disk into working memory (or RAM). We also allocate a bit of space in RAM for the process to store variables (to hold temporary working data) and a few flags to allow the operating system (OS) to manage and track the process during its execution.

Essentially a process is a running instance of a program.

There could be several processes representing the same program running in memory at the same time. For example I could have two terminals open and be running the command `cp` in both of them. In this case there would be two `cp` processes currently existing on the system. Once they are finished running the system then destroys them and there are no longer any processes representing the program `cp`.

When we are at the terminal we have a Bash process running in order to give us the Bash shell. If we start a script running it doesn't actually run in that process but instead starts a new process to run inside. We'll demonstrate this in the next section on variables and its implications should become clearer. For the most part you don't need to worry too much about this phenomenon however.

How do we run them?

Running a Bash script is fairly easy. Another term you may come across is **executing** the script (which means the same thing). Before we can execute a script it must have the execute permission set (for safety reasons this permission is generally not set by default). If you forget to grant this permission before running the script you'll just get an error message telling you as such and no harm will be done.

Terminal

```
1. user@bash: ./myscript.sh
2. bash: ./myscript.sh: Permission denied
3. user@bash: ls -l myscript.sh
```

```
4. -rw-r--r-- 18 ryan users 4096 Feb 17 09:12 myscript.sh
5. user@bash: chmod 755 myscript.sh
6. user@bash: ls -l myscript.sh
7. -rwxr-xr-x 18 ryan users 4096 Feb 17 09:12 myscript.sh
8. user@bash: ./myscript.sh
9. Hello World!
10. user@bash:
```

Tip

The shorthand 755 is often used for scripts as it allows you the owner to write or modify the script and for everyone to execute the script.

Here are the contents of myscript.sh

myscript.sh	
1.	#!/bin/bash
2.	<i># A sample Bash script, by Ryan</i>
3.	
4.	echo Hello World!

Let's break it down:

- **Line 1** - Is what's referred to as the **shebang**. See below for what this is.
- **Line 2** - This is a comment. Anything after **#** is not executed. It is for our reference only.
- **Line 4** - Is the command **echo** which will print a message to the screen. You can type this command yourself on the command line and it will behave exactly the same.
- The syntax highlighting is there only to make it easier to read and is not something you need to do in your own files (remember they are just plain text files).

Why the ./

You've possibly noticed that when we run a normal command (such as **ls**) we just type its name but when running the script above I put a **./** in front of it. When you just type a name on the command line Bash tries to find it in a series of directories stored in a variable called **\$PATH**. We can see the current value of this variable using the command **echo** (you'll learn more about variables in the next section).

Terminal	
1.	user@bash: echo \$PATH
2.	/home/ryan/bin:/usr/local/bin:/usr/bin:/bin
3.	user@bash:

The directories are separated by " : "

Bash only looks in those specific directories and doesn't consider sub directories or your current directory. It will look through those directories in order and execute the first instance of the program or script that it finds.

The **\$PATH** variable is an individual user variable so each user on a system may set it to suit themselves.

This is done for a few different reasons.

- It allows us to have several different versions of a program installed. We can control which one gets executed based on where it sits in our **\$PATH**.
- It allows for convenience. As you saw above, the first directory for myself is a **bin** directory in my home directory. This allows me to put my own scripts and programs there and then I can use them no matter where I am in the system by just typing their name. I could even create a script with the same name as a program (to act as a wrapper) if I wanted slightly different behaviour.
- It increases safety - For example a malicious user could create a script called **ls** which actually deletes everything in your home directory. You wouldn't want to inadvertently run that script. But as long as it's not in your **\$PATH** that won't happen.

If a program or script is not in one of the directories in your **\$PATH** then you can run it by telling Bash where it should look to find it. You do so by including either an absolute or relative path in front of the program or script name. You'll remember that dot (**.**) is actually a reference to your current directory. Assuming this script is in my home directory I could also have run it by using an absolute path.

Terminal	
1.	user@bash: /home/ryan/myscript.sh
2.	Hello World!
3.	user@bash:

The Shebang (#!)

#!/bin/bash

This is the first line of the script above. The hash exclamation mark (**#!**) character sequence is referred to as the Shebang. Following it is the path to the interpreter (or program) that should be used to run (or interpret) the rest of the lines in the text file. (For Bash

scripts it will be the path to Bash, but there are many other types of scripts and they each have their own interpreter.)

Formatting is important here. The shebang must be on the very first line of the file (line 2 won't do, even if the first line is blank). There must also be no spaces before the # or between the ! and the path to the interpreter.

Whilst you could use a relative path for the interpreter, most of the time you are going to want to use an absolute path. You will probably be running the script from a variety of locations so absolute is the safest (and often shorter than a relative path too in this particular case).

It is possible to leave out the line with the shebang and still run the script but it is unwise. If you are at a terminal and running the Bash shell and you execute a script without a shebang then Bash will assume it is a Bash script. So this will only work assuming the user running the script is running it in a Bash shell and there are a variety of reasons why this may not be the case, which is dangerous.

You can also run Bash, passing the script as an argument.

```
Terminal
1. user@bash: bash myscript.sh
2. Hello World!
3. user@bash:
```

Whilst this is safe it also involves unnecessary typing every time you want to run the script.

Best Practice

Given the observations above it is best to always include the shebang (#!). It is the most reliable and convenient approach.

Formatting

As we saw above, formatting for the shebang was important (ie no spaces, must be on first line). There are many areas in Bash scripts where formatting is important. Typically it involves spaces and either the presence or absence of a space can be the difference between the command working or not. I'll point these out as we encounter them. Also get in the habit of being mindful of the presence or absence of spaces when looking at code.

The main reason for this is that Bash was originally developed as an interface for Users to interact with the system and later extended to have more powerful scripting capabilities. Many decisions regarding it's behaviour were made considering only the needs of the user and then scripting capabilities had to be worked in, later, around those decisions. People generally don't mind this however as Bash scripts are still an awesome tool for quickly and easily joining existing programs into more powerful solutions.

I have seen students spend quite a bit of time in frustration that a piece of code that looks perfectly fine isn't working. They get quite embarrassed when they find out the culprit (a space that either should or shouldn't be there). You will probably make this mistake a few times yourself before it sinks in so don't worry too much but the sooner you get the hang of it the happier you will be :)

Indenting of code is another area of formatting that is important. We'll look at [indenting of code](#) in section 5 ([If Statements](#)) when it becomes relevant. Indenting is not required but it does make your code easier to read and make it harder to make simple errors.

[Variables](#) ➤

Variables!

Temporary stores of information



SECTION BREAKDOWN

- [Introduction](#)
- [Command Line Arguments](#)
- [Other Special Variables](#)
- [Setting Our Own Variables](#)
- [Quotes](#)
- [Command Substitution](#)
- [Exporting Variables](#)
- [Summary](#)
- [Activities](#)
- NEXT SECTION
- [Input](#)

Introduction

For those of you that have dabbled in programming before, you'll be quite familiar with variables. For those of you that haven't, think of a variable as a temporary store for a simple piece of information. These variables can be very useful for allowing us to manage and control the actions of our Bash Script. We'll go through a variety of different ways that variables have their data set and ways we can then use them.

Variables are one of those things that are actually quite easy to use but are also quite easy to get yourself into trouble with if you don't properly understand how they work. As such there is a bit of reading in this section but if you take the time to go through and understand it you will be thankful you did later on when we start dabbling in more complex scripts.

How do they Work?

A variable is a temporary store for a piece of information. There are two actions we may perform for variables:

- Setting a value for a variable.
- Reading the value for a variable.

Variables may have their value set in a few different ways. The most common are to set the value directly and for its value to be set as the result of processing by a command or program. You will see examples of both below.

To read the variable we then place its name (preceded by a \$ sign) anywhere in the script we would like. Before Bash interprets (or runs) every line of our script it first checks to see if any variable names are present. For every variable it has identified, it replaces the variable name with its value. Then it runs that line of code and begins the process again on the next line.

Here are a few quick points on syntax. They will be elaborated on and demonstrated as we go into more detail below.

- When referring to or reading a variable we place a \$ sign before the variable name.
- When setting a variable we leave out the \$ sign.
- Some people like to always write variable names in uppercase so they stand out. It's your preference however. They can be all uppercase, all lowercase, or a mixture.
- A variable may be placed anywhere in a script (or on the command line for that matter) and, when run, Bash will replace it with the value of the variable. This is made possible as the substitution is done before the command is run.

Command Line arguments

Command line arguments are commonly used and easy to work with so they are a good place to start.

When we run a program on the command line you would be familiar with supplying arguments after it to control its behaviour. For instance we could run the command `ls -l /etc`. `-l` and `/etc` are both command line arguments to the command `ls`. We can do similar with our bash scripts. To do this we use the variables `$1` to represent the first command line argument, `$2` to represent the second command line argument and so on. These are automatically set by the system when we run our script so all we need to do is refer to them.

Let's look at an example.

```

mycopy.sh
1.  #!/bin/bash
2.  # A simple copy script
3.
4.  cp $1 $2
5.
6.  # Let's verify the copy worked
7.
8.  echo Details for $2
9.  ls -lh $2

```

Let's break it down:

- **Line 4** - run the command `cp` with the first command line argument as the source and the second command line argument as the destination.

- **Line 8** - run the command **echo** to print a message.
- **Line 9** - After the copy has completed, run the command **ls** for the destination just to verify it worked. We have included the options **l** to show us extra information and **h** to make the size human readable so we may verify it copied correctly.

```
Terminal
1. user@bash: ./mycopy.sh /projects/file1.data ./results.data
2. Details for ./results.data
3. -rw-r--r-- 18 ryan users 3.4M Feb 14 07:18 results.data
4. user@bash:
```

We'll discuss their usage a little more in the next section ([3. Input](#)).

Other Special Variables

There are a few other variables that the system sets for you to use as well.

- **\$0** - The name of the Bash script.
- **\$1 - \$9** - The first 9 arguments to the Bash script. (As mentioned above.)
- **\$#** - How many arguments were passed to the Bash script.
- **\$@** - All the arguments supplied to the Bash script.
- **\$?** - The exit status of the most recently run process.
- **\$\$** - The process ID of the current script.
- **\$USER** - The username of the user running the script.
- **\$HOSTNAME** - The hostname of the machine the script is running on.
- **\$SECONDS** - The number of seconds since the script was started.
- **\$RANDOM** - Returns a different random number each time it is referred to.
- **\$LINENO** - Returns the current line number in the Bash script.

Tip

If you type the command **env** on the command line you will see a listing of other variables which you may also refer to.

Some of these variables may seem useful to you now. Others may not. As we progress to more complex scripts in later sections you will see examples of how they can be useful.

Setting Our Own Variables

As well as variables that are preset by the system, we may also set our own variables. This can be useful for keeping track of results of commands and being able to refer to and process them later.

There are a few ways in which variables may be set (such as part of the execution of a command) but the basic form follows this pattern:

```
variable=value
```

This is one of those areas where formatting is important. Note there is no space on either side of the equals (=) sign. We also leave off the \$ sign from the beginning of the variable name when setting it.

Variable names may be uppercase or lowercase or a mixture of both but Bash is a case sensitive environment so whenever you refer to a variable you must be consistent in your use of uppercase and lowercase letters. You should always make sure variable names are descriptive. This makes their purpose easier for you to remember.

Here is a simple example to illustrate their usage.

```
simplevariables.sh
1. #!/bin/bash
2. # A simple variable example
3.
4. myvariable=Hello
5.
6. anothervar=Fred
7.
8. echo $myvariable $anothervar
9. echo
10.
11. sampledir=/etc
12.
13. ls $sampledir
```

Let's break it down:

- **Lines 4 and 6** - set the value of the two variables **myvariable** and **anothervar**.
- **Line 8** - run the command **echo** to check the variables have been set as intended.
- **Line 9** - run the command **echo** this time with no arguments. This is a good way to get a blank line on the screen to help space things out.

- **Line 11** - set another variable, this time as the path to a particular directory.
- **Line 13** - run the command **ls** substituting the value of the variable `sampledir` as its first command line argument.

```
Terminal
1. user@bash: ./simplevariables.sh
2. Hello Fred
3.
4. a2ps.cfg aliases alsa.d ...
5. user@bash:
```

It is important to note that in the example above we used the command **echo** simply because it is a convenient way to demonstrate that the variables have actually been set. `echo` is not needed to make use of variables and is only used when you wish to print a specific message to the screen. (Pretty much all commands print output to the screen as default so you don't need to put `echo` in front of them.)

Best Practice

Variables can be useful for making our scripts easier to manage. Maybe our script is going to run several commands, several of which will refer to a particular directory. Rather than type that directory out each time we can set it once in a variable then refer to that variable. Then if the required directory changes in the future we only need to update one variable rather than every instance within the script.

Quotes

In the example above we kept things nice and simple. The variables only had to store a single word. When we want variables to store more complex values however, we need to make use of quotes. This is because under normal circumstances Bash uses a space to determine separate items.

```
Terminal
1. user@bash: myvar=Hello World
2. -bash: World: command not found
3. user@bash:
```

- Remember, commands work exactly the same on the command line as they do within a script.

Tip

Because commands work exactly the same on the command line as in a script it can sometimes be easier to experiment on the command line.

When we enclose our content in quotes we are indicating to Bash that the contents should be considered as a single item. You may use single quotes (`'`) or double quotes (`"`).

- Single quotes will treat every character literally.
- Double quotes will allow you to do substitution (that is include variables within the setting of the value).

```
Terminal
1. user@bash: myvar='Hello World'
2. user@bash: echo $myvar
3. Hello World
4. user@bash: newvar="More $myvar"
5. user@bash: echo $newvar
6. More Hello World
7. user@bash: newvar='More $myvar'
8. user@bash: echo $newvar
9. More $myvar
10. user@bash:
```

Command Substitution

Command substitution allows us to take the output of a command or program (what would normally be printed to the screen) and save it as the value of a variable. To do this we place it within brackets, preceded by a `$` sign.

```
Terminal
1. user@bash: myvar=$( ls /etc | wc -l )
2. user@bash: echo There are $myvar entries in the directory /etc
3. user@bash:
```

Command substitution is nice and simple if the output of the command is a single word or line. If the output goes over several lines then the newlines are simply removed and all the output ends up on a single line.

```
Terminal
1. user@bash: ls
2. bin Documents Desktop ...
3. Downloads public_html ...
4. user@bash: myvar=$( ls )
5. user@bash: echo $myvar
6. bin Documents Desktop Downloads public_html ...
7. user@bash:
```


Let's break it down:

- **Line 1** - We run the command **ls**. Normally its output would be over several lines. I have shortened it a bit in the example above just to save space.
- **Line 4** - When we save the command to the variable **myvar** all the newlines are stripped out and the output is now all on a single line.

Tip

When playing about with command substitution it's a good idea to test your output rather than just assuming it will behave in a certain way. The easiest way to do that is simply to **echo** the variable and see what has happened. (You can then remove the echo command once you are happy.)

Exporting Variables

Remember how in the previous section we talked about [scripts being run in their own process](#)? This introduces a phenomenon known as scope which affects variables amongst other things. The idea is that variables are limited to the process they were created in. Normally this isn't an issue but sometimes, for instance, a script may run another script as one of its commands. If we want the variable to be available to the second script then we need to export the variable.

```
script1.sh
1.  #!/bin/bash
2.  # demonstrate variable scope 1.
3.
4.  var1=blah
5.  var2=foo
6.
7.  # Let's verify their current value
8.
9.  echo $0 :: var1 : $var1, var2 : $var2
10.
11. export var1
12. ./script2.sh
13.
14. # Let's see what they are now
15.
16. echo $0 :: var1 : $var1, var2 : $var2
```

```
script2.sh
1.  #!/bin/bash
2.  # demonstrate variable scope 2
3.
4.  # Let's verify their current value
5.
6.  echo $0 :: var1 : $var1, var2 : $var2
7.
8.  # Let's change their values
9.
10. var1=flop
11. var2=bleh
12.
```

Now lets run it and see what happens.

```
Terminal
1.  user@bash: ./script1.sh
2.  script1.sh :: var1 : blah, var2 : foo
3.  script2.sh :: var1 : blah, var2 :
4.  script1.sh :: var1 : blah, var2 : foo
5.  user@bash:
```

The output above may seem unexpected. What actually happens when we export a variable is that we are telling Bash that every time a new process is created (to run another script or such) then make a copy of the variable and hand it over to the new process. So although the variables will have the same name they exist in separate processes and so are unrelated to each other.

Exporting variables is a one way process. The original process may pass variables over to the new process but anything that process does with the copy of the variables has no impact on the original variables.

Exporting variables is something you probably won't need to worry about for most Bash scripts you'll create. Sometimes you may wish to break a particular task down into several separate scripts however to make it easier to manage or to allow for reusability (which is always good). For instance you could create a script which will make a dated (ie todays date prepended to the filename) copy of all filenames exported on a certain variable. Then you could easily call that script from within other scripts you create whenever you would like to take a snapshot of a set of files.

Summary

Stuff We Learnt

\$1, \$2, ...

The first, second, etc command line arguments to the script.

variable=value

To set a value for a variable. Remember, no spaces on either side of =

Quotes " ' '

Double will do variable substitution, single will not.

variable=\$(command)

Save the output of a command into a variable

export var1

Make the variable var1 available to child processes.

Important Concepts

Formatting

The presence or absence of spaces is important.

Manageability

If a particular value is used several times within a script (eg a file or directory name) then using a variable can make it easier to manage.

Activities

Let's explore variables.

- A good place to start is to create a simple script which will accept some command line arguments and **echo** out some details about them (eg, how many are there, what is the secone one etc).
- Create a script which will print a random word. There is a file containing a list of words on your system (usually /usr/share/dict/words or /usr/dict/words). Hint: [Piping](#) will be useful here.
- Expand the previous activity so that if a number is supplied as the first command line argument then it will select from only words with that many characters. Hint: [Grep](#) may be useful here.
- Take a copy of the two files script1.sh and script2.sh above then experiment by tweaking them and running them and observing the output. This will help you get a feel for how exporting variables works.
- Now let's create a script which will take a filename as its first argument and create a dated copy of the file. eg. if our file was named file1.txt it would create a copy such as 2021-12-23_file1.txt. (To achieve this you will probably want to play with command substitution and the command **date**)
- **Challenge:** To make it a bit harder, see if you can get it so that the date is after the name of the file (eg. file1_2021-12-23.txt (The command **basename** can be useful here.)
- **Challenge:** Now see if you can expand the previous question to accept a list of files on the command line and it will create a named copy of all of them. (The command [xargs](#) may be useful here.)

Input ➤

User Input!

Let's make our scripts interactive.



SECTION BREAKDOWN

[Introduction](#)
[Ask User for Input](#)
[Reading from STDIN](#)
[So Which Should I Use?](#)
[Summary](#)
[Activities](#)
[NEXT SECTION](#)
[Arithmetic](#)

Introduction

We looked at one form of user input ([command line arguments](#)) in the previous section. Now we would like to introduce other ways the user may provide input to the Bash script. Following this we'll have a discussion on when and where is best to use each method.

After the mammoth previous section this one is much easier to get through.

Ask the User for Input

If we would like to ask the user for input then we use a command called **read**. This command takes the input and will save it into a variable.

```
read var1
```

Let's look at a simple example:

```

introduction.sh
1.  #!/bin/bash
2.  # Ask the user for their name
3.
4.  echo Hello, who am I talking to?
5.
6.  read varname
7.
8.  echo It\'s nice to meet you $varname

```

Let's break it down:

- **Line 4** - Print a message asking the user for input.
- **Line 6** - Run the command **read** and save the users response into the variable **varname**
- **Line 8** - **echo** another message just to verify the read command worked. Note: I had to put a backslash (\) in front of the ' so that it was escaped.

```

Terminal
1.  user@bash: ./introduction.sh
2.  Hello, who am I talking to?
3.  Ryan
4.  It's nice to meet you Ryan
5.  user@bash:

```

- Note: Ryan above is in italics just to show that it was something I typed in. On your terminal input will show up normally.

More with Read

You are able to alter the behaviour of **read** with a variety of command line options. (See the man page for read to see all of them.) Two commonly used options however are **-p** which allows you to specify a prompt and **-s** which makes the input silent. This can make it easy to ask for a username and password combination like the example below:

```

login.sh
1.  #!/bin/bash
2.  # Ask the user for login details
3.
4.  read -p 'Username: ' uservar
5.  read -sp 'Password: ' passvar
6.  echo
7.  echo Thankyou $uservar we now have your login details

```

- On lines 4 and 5 above we include the prompt within quotes so we can have a space included with it. Otherwise the user input will start straight after the last character of the prompt which isn't ideal from a readability point of view.

```
Terminal
1. user@bash: ./login.sh
2. Username: ryan
3. Password:
4. Thankyou ryan we now have your login details
5. user@bash:
```

More variables

So far we have looked at a single word as input. We can do more than that however.

```
cars.sh
1. #!/bin/bash
2. # Demonstrate how read actually works
3.
4. echo What cars do you like?
5.
6. read car1 car2 car3
7.
8. echo Your first car was: $car1
9. echo Your second car was: $car2
10. echo Your third car was: $car3
```

```
Terminal
1. user@bash: ./cars.sh
2. What cars do you like?
3. Jaguar Maserati Bentley
4. Your first car was: Jaguar
5. Your second car was: Maserati
6. Your third car was: Bentley
7. user@bash:
8. user@bash: ./cars.sh
9. What cars do you like?
10. Jaguar Maserati Bentley Lotus
11. Your first car was: Jaguar
12. Your second car was: Maserati
13. Your third car was: Bentley Lotus
14. user@bash:
```

The general mechanism is that you can supply several variable names to **read**. Read will then take your input and split it on whitespace. The first item will then be assigned to the first variable name, the second item to the second variable name and so on. If there are more items than variable names then the remaining items will all be added to the last variable name. If there are less items than variable names then the remaining variable names will be set to blank or null.

Reading from STDIN

It's common in Linux to **pipe** a series of simple, single purpose commands together to create a larger solution tailored to our exact needs. The ability to do this is one of the real strengths of Linux. It turns out that we can easily accommodate this mechanism with our scripts also. By doing so we can create scripts that act as filters to modify data in specific ways for us.

Bash accommodates piping and redirection by way of special files. Each process gets it's own set of files (one for STDIN, STDOUT and STDERR respectively) and they are linked when piping or redirection is invoked. Each process gets the following files:

- **STDIN** - /proc/<processID>/fd/0
- **STDOUT** - /proc/<processID>/fd/1
- **STDERR** - /proc/<processID>/fd/2

To make life more convenient the system creates some shortcuts for us:

- **STDIN** - /dev/stdin or /proc/self/fd/0
- **STDOUT** - /dev/stdout or /proc/self/fd/1
- **STDERR** - /dev/stderr or /proc/self/fd/2

fd in the paths above stands for file descriptor.

So if we would like to make our script able to process data that is piped to it all we need to do is read the relevant file. All of the files mentioned above behave like normal files.

```
summary
1. #!/bin/bash
2. # A basic summary of my sales report
3.
4. echo Here is a summary of the sales data:
5. echo =====
6. echo
7.
8. cat /dev/stdin | cut -d' ' -f 2,3 | sort
```

Let's break it down:

- **Lines 4, 5, 6** - Print a title for the output
- **Line 8** - **cat** the file representing STDIN, **cut** setting the delimiter to a space, fields 2 and 3 then sort the output.

```
Terminal
1. user@bash: cat salesdata.txt
2. Fred apples 20 December 4
3. Susy oranges 5 December 7
4. Mark watermelons 12 December 10
5. Terry peaches 7 December 15
6. user@bash:
7. user@bash: cat salesdata.txt | ./summary
8. Here is a summary of the sales data:
9. =====
10.
11. apples 20
12. oranges 5
13. peaches 7
14. watermelons 12
15. user@bash:
```

So which should I use?

So we now have 3 methods for getting input from the user:

- Command line arguments
- Read input during script execution
- Accept data that has been redirected into the Bash script via STDIN

Which method is best depends on the situation.

You should normally favor command line arguments wherever possible. They are the most convenient for users as the data will be stored in their command history so they can easily return to it. It is also the best approach if your script may be called by other scripts or processes (eg. maybe you want it to run periodically using CRON).

Sometimes the nature of the data is such that it would not be ideal for it to be stored in peoples command histories etc. A good example of this is login credentials (username and password). In these circumstances it is best to **read** the data during script execution.

If all the script is doing is processing data in a certain way then it is probably best to work with STDIN. This way it can easily be added into a pipeline.

Sometimes you may find that a combination is ideal. The user may supply a filename as a command line argument and if not then the script will process what it finds on STDIN (when we look at If statements we'll see how this may be achieved). Or maybe command line arguments define certain behaviour but **read** is also used to ask for more information if required.

Ultimately you should think about 3 factors when deciding how users will supply data to your Bash script:

- **Ease of use** - which of these methods will make it easiest for users to use my script?
- **Security** - Is there sensitive data which I should handle appropriately?
- **Robustness** - Can I make it so that my scripts operation is intuitive and flexible and also make it harder to make simple mistakes?

Summary

Stuff We Learnt

read varName

Read input from the user and store it in the variable varName.

/dev/stdin

A file you can read to get the STDIN for the Bash script

Important Concepts

Usability

Your choice of input methods will have an impact on how useable your script is.

Activities

Let's dabble with input.

- Create a simple script which will ask the user for a few pieces of information then combine this into a message which is echo'd to the screen.
- Add to the previous script to add in some data coming from command line arguments and maybe some of the other system variables.
- Create a script which will take data from STDIN and print the 3rd line only.
- Now play about with creating a script which will behave as a filter. Create a script which will rearrange the output of the command **ls -l** in a useful way (eg maybe you only print the filename, size and owner) (Hint: [awk](#) can be useful here).



SECTION BREAKDOWN

[Introduction](#)

[Let](#)

[Expr](#)

[Double Parentheses](#)

[Length of a Variable](#)

[Summary](#)

[Activities](#)

NEXT SECTION

[If Statements](#)

Introduction

Depending on what type of work you want your scripts to do you may end up using arithmetic a lot or not much at all. It's a reasonable certainty however that you will need to use arithmetic at some point. Like variables, they are reasonably easy to implement and knowing how to do so is an essential skill in Bash scripting mastery.

There are several ways to go about arithmetic in Bash scripting. We'll cover them for completeness but the recommended approach is arithmetic expansion (covered last).

Let

`let` is a builtin function of Bash that allows us to do simple arithmetic. It follows the basic format:

```
let <arithmetic expression>
```

The arithmetic expression can take a variety of formats which we'll outline below. The first part is generally always a variable which the result is saved into however.

Let's look at a simple example:

```
let_example.sh
1.  #!/bin/bash
2.  # Basic arithmetic using let
3.
4.  let a=5+4
5.  echo $a # 9
6.
7.  let "a = 5 + 4"
8.  echo $a # 9
9.
10. let a++
11. echo $a # 10
12.
13. let "a = 4 * 5"
14. echo $a # 20
15.
16. let "a = $1 + 30"
17. echo $a # 30 + first command line argument
```

Let's break it down:

- **Line 4** - This is the basic format. Note that if we don't put quotes around the expression then it must be written with no spaces.
- **Line 7** - This time we have used quotes which allow us to space out the expression to make it more readable.
- **Line 10** - This is a shorthand for increment the value of the variable `a` by 1. It is the same as writing `"a = a + 1"`.
- **Line 16** - We may also include other variables in the expression.

```
Terminal
1.  user@bash: ./let_example.sh 15
2.  9
3.  9
4.  10
5.  20
6.  45
7.  user@bash:
```

Here is a table with some of the basic expressions you may perform. There are others but these are the most commonly used.

Operator	Operation
<code>+, -, *, /</code>	addition, subtraction, multiply, divide

var++	Increase the variable var by 1
var--	Decrease the variable var by 1
%	Modulus (Return the remainder after division)

These operators may be used in the other mechanisms described below as well.

Expr

expr is similar to **let** except instead of saving the result to a variable it instead prints the answer. Unlike **let** you don't need to enclose the expression in quotes. You also must have spaces between the items of the expression. It is also common to use **expr** within command substitution to save the output to a variable.

```
expr item1 operator item2
```

Let's look at a simple example:

```
expr_example.sh
1.  #!/bin/bash
2.  # Basic arithmetic using expr
3.
4.  expr 5 + 4
5.
6.  expr "5 + 4"
7.
8.  expr 5+4
9.
10. expr 5 \* $1
11.
12. expr 11 % 2
13.
14. a=$( expr 10 - 3 )
15. echo $a # 7
```

Let's break it down:

- **Line 4** - This is the basic format. Note that there must be spaces between the items and no quotes.
- **Line 6** - If we do put quotes around the expression then the expression will not be evaluated but printed instead.
- **Line 8** - If we do not put spaces between the items of the expression then the expression will not be evaluated but printed instead.
- **Line 10** - Some characters have a special meaning to Bash so we must escape them (put a backslash in front of) to remove their special meaning.
- **Line 12** - Here we demonstrate the operator **modulus**. The modulus is the remainder when the first item is divided by the second item.
- **Line 14** - This time we're using **expr** within command substitution in order to save the result to the variable **a**.

```
Terminal
1.  user@bash: ./expr_example.sh 12
2.  9
3.  5 + 4
4.  5+4
5.  60
6.  1
7.  7
8.  user@bash:
```

Double Parentheses

In the section on Variables we saw that we could save the output of a command easily to a variable. It turns out that this mechanism is also able to do basic arithmetic for us if we tweak the syntax a little. We do so by using double brackets like so:

```
$( ( expression ) )
```

Here's an example to illustrate:

```
expansion_example.sh
1.  #!/bin/bash
2.  # Basic arithmetic using double parentheses
3.
4.  a=$(( 4 + 5 ))
5.  echo $a # 9
6.
7.  a=$((3+5))
```

```

8. echo $a # 8
9.
10. b=$(( a + 3 ))
11. echo $b # 11
12.
13. b=$(( $a + 4 ))
14. echo $b # 12
15.
16. (( b++ ))
17. echo $b # 13
18.
19. (( b += 3 ))
20. echo $b # 16
21.
22. a=$(( 4 * 5 ))
23. echo $a # 20

```

Let's break it down:

- **Line 4** - This is the basic format. As you can see we may space it out nicely for readability without the need for quotes.
- **Line 7** - As you can see, it works just the same if we take spacing out.
- **Line 10** - We may include variables without the preceding \$ sign.
- **Line 13** - Variables can be included with the \$ sign if you prefer.
- **Line 16** - This is a slightly different form. Here the value of the variable b is incremented by 1 (using the same mechanism illustrated under **let**). When we do this we don't need the \$ sign preceding the brackets.
- **Line 19** - This is a slightly different form of the previous example. Here the value of the variable b is incremented by 3. It is a shorthand for **b = b + 3**.
- **Line 19** - Unlike other methods, when we do multiplication we don't need to escape the * sign.

```

Terminal
1. user@bash: ./expansion_example.sh
2. 9
3. 8
4. 11
5. 12
6. 13
7. 16
8. 20
9. user@bash:

```

So as you can see double parentheses is quite flexible in how you format it's expression. This is part of why we prefer this method. As double parentheses is builtin to Bash it also runs slightly more efficiently (though to be honest, with the raw computing power of machines these days the difference in performance is really insignificant).

Length of a Variable

This isn't really arithmetic but it can be quite useful. If you want to find out the length of a variable (how many characters) you can do the following:

```

${#variable}

```

Here's an example:

```

length_example.sh
1. #!/bin/bash
2. # Show the Length of a variable.
3.
4. a='Hello World'
5. echo ${#a} # 11
6.
7. b=4953
8. echo ${#b} # 4

```

```

Terminal
1. user@bash: ./length_example.sh
2. 11
3. 4
4. user@bash:

```

Summary

Stuff We Learnt

let expression

Make a variable equal to an expression.

expr expression

print out the result of the expression.

\$((expression))

Return the result of the expression.


```
#{#var}  
Return the length of the variable var.
```

Important Concepts

Arithmetic

There are several ways in which to do arithmetic in Bash scripts. Double parentheses is the preferred method.

Formatting

When doing arithmetic, the presence or absence of spaces (and quotes) is often important.

Activities

Let's dive in with arithmetic.

- Create a simple script which will take two command line arguments and then multiply them together using each of the methods detailed above.
- Write a Bash script which will print tomorrows date. (Hint: use the command **date**)
- Remember when we looked at [variables](#) we discovered \$RANDOM will return a random number. This number is between 0 and 32767 which is not always the most useful. Let's write a script which will use this variable and some arithmetic (hint: play with modulus) to return a random number between 0 and 100.
- Now let's play with the previous script. Modify it so that you can specify as a command line argument the upper limit of the random number. Can you make it so that a lower limit can be specified also? eg. if I ran ./random.sh 10 45 it would only return random numbers between 10 and 45.

[If Statements](#) ➤

If Statements!

Decisions, decisions.



SECTION BREAKDOWN

- [Introduction](#)
- [Basic If Statements](#)
- [Test](#)
- [Indenting](#)
- [Nested If Statements](#)
- [If Else](#)
- [If Elif Else](#)
- [Boolean Operations](#)
- [Case Statements](#)

Summary

[Activities](#)[NEXT SECTION](#)[Loops](#)

Introduction

Bash if statements are very useful. In this section of our Bash Scripting Tutorial you will learn the ways you may use if statements in your Bash scripts to help automate tasks.

If statements (and, closely related, **case** statements) allow us to make decisions in our Bash scripts. They allow us to decide whether or not to run a piece of code based upon conditions that we may set. If statements, combined with [loops](#) (which we'll look at in the next section) allow us to make much more complex scripts which may solve larger tasks.

Like what we have looked at in previous sections, their syntax is very specific so stay on top of all the little details.

Basic If Statements

A basic if statement effectively says, **if** a particular test is true, then perform a given set of actions. If it is not true then don't perform those actions. It follows the format below:

```
if [ <some test> ]
then
  <commands>
fi
```

Anything between **then** and **fi** (if backwards) will be executed only if the test (between the square brackets) is true.

Let's look at a simple example:

```
if_example.sh
1.  #!/bin/bash
2.  # Basic if statement
3.
4.  if [ $1 -gt 100 ]
5.  then
6.    echo Hey that's a large number.
7.    pwd
8.  fi
9.
10. date
```

Let's break it down:

- **Line 4** - Let's see if the first command line argument is greater than 100
- **Line 6 and 7** - Will only get run if the test on line 4 returns true. You can have as many commands here as you like.
- **Line 6** - The backslash (\) in front of the single quote (') is needed as the single quote has a special meaning for bash and we don't want that special meaning. The backslash escapes the special meaning to make it a normal plain single quote again.
- **Line 8** - fi signals the end of the if statement. All commands after this will be run as normal.
- **Line 10** - Because this command is outside the if statement it will be run regardless of the outcome of the if statement.

```
Terminal
1.  user@bash: ./if_example.sh 15
2.  Thu 23 Dec 4:09:19 2021
3.  user@bash: ./if_example.sh 150
4.  Hey that's a large number.
5.  /home/ryan/bin
6.  Thu 23 Dec 4:09:19 2021
7.  user@bash:
```

Tip

It is always good practice to test your scripts with input that covers the different scenarios that are possible.

Test

The square brackets (`[]`) in the `if` statement above are actually a reference to the command `test`. This means that all of the operators that `test` allows may be used here as well. Look up the man page for `test` to see all of the possible operators (there are quite a few) but some of the more common ones are listed below.

Operator	Description
<code>! EXPRESSION</code>	The EXPRESSION is false.
<code>-n STRING</code>	The length of STRING is greater than zero.
<code>-z STRING</code>	The length of STRING is zero (ie it is empty).
<code>STRING1 = STRING2</code>	STRING1 is equal to STRING2
<code>STRING1 != STRING2</code>	STRING1 is not equal to STRING2
<code>INTEGER1 -eq INTEGER2</code>	INTEGER1 is numerically equal to INTEGER2
<code>INTEGER1 -gt INTEGER2</code>	INTEGER1 is numerically greater than INTEGER2
<code>INTEGER1 -lt INTEGER2</code>	INTEGER1 is numerically less than INTEGER2
<code>-d FILE</code>	FILE exists and is a directory.
<code>-e FILE</code>	FILE exists.
<code>-r FILE</code>	FILE exists and the read permission is granted.
<code>-s FILE</code>	FILE exists and its size is greater than zero (ie. it is not empty).
<code>-w FILE</code>	FILE exists and the write permission is granted.
<code>-x FILE</code>	FILE exists and the execute permission is granted.

A few points to note:

- `=` is slightly different to `-eq`. `[ 001 = 1 ]` will return false as `=` does a string comparison (ie. character for character the same) whereas `-eq` does a numerical comparison meaning `[ 001 -eq 1 ]` will return true.
- When we refer to **FILE** above we are actually meaning a [path](#). Remember that a path may be absolute or relative and may refer to a file or a directory.
- Because `[]` is just a reference to the command `test` we may experiment and trouble shoot with `test` on the command line to make sure our understanding of its behaviour is correct.

```
Terminal
1. user@bash: test 001 = 1
2. user@bash: echo $?
3. 1
4. user@bash: test 001 -eq 1
5. user@bash: echo $?
6. 0
7. user@bash: touch myfile
8. user@bash: test -s myfile
9. user@bash: echo $?
10. 1
11. user@bash: ls /etc > myfile
12. user@bash: test -s myfile
13. user@bash: echo $?
14. 0
15. user@bash:
```

Let's break it down:

- **Line 1** - Perform a string based comparison. `test` doesn't print the result so instead we check its exit status which is what we will do on the next line.
- **Line 2** - The variable `$?` holds the exit status of the previously run command (in this case `test`). 0 means TRUE (or success). 1 = FALSE (or failure).
- **Line 4** - This time we are performing a numerical comparison.
- **Line 7** - Create a new blank file **myfile** (assuming that **myfile** doesn't already exist).
- **Line 8** - Is the size of **myfile** greater than zero?
- **Line 11** - [Redirect](#) some content into **myfile** so its size is greater than zero.
- **Line 12** - Test the size of **myfile** again. This time it is TRUE.

Indenting

You'll notice that in the `if` statement above we indented the commands that were run if the statement was true. This is referred to as indenting and is an important part of writing good, clean code (in any language, not just Bash scripts). The aim is to improve readability and make it harder for us to make simple, silly mistakes. There aren't any rules regarding indenting in Bash so you may indent or not indent however you like and your scripts will still run exactly the same. I would highly recommend you do indent your code however (especially as your scripts get larger) otherwise you will find it increasingly difficult to see the structure in your scripts.

Nested If statements

Talking of indenting. Here's a perfect example of when it makes life easier for you. You may have as many `if` statements as necessary inside your script. It is also possible to have an `if` statement inside of another `if` statement. For example, we may want to analyse a number given on the command line like so:

```
nested_if.sh
```

```

1.  #!/bin/bash
2.  # Nested if statements
3.
4.  if [ $1 -gt 100 ]
5.  then
6.      echo Hey that's a large number.
7.
8.      if (( $1 % 2 == 0 ))
9.      then
10.         echo And is also an even number.
11.     fi
12. fi

```

Let's break it down:

- **Line 4** - Perform the following, only if the first command line argument is greater than 100.
- **Line 8** - This is a light variation on the **if** statement. If we would like to check an expression then we may use the double brackets just like we did for [variables](#).
- **Line 10** - Only gets run if both if statements are true.

“

Yo dawg, I herd you like **if** statements so I put an **if** statement inside your **if** statement.

Xzibit

([Xzibit](#) didn't actually say that but I'm sure he would have, had he hosted Pimp My Bash Script.)

Tip

You can nest as many if statements as you like but as a general rule of thumb if you need to nest more than 3 levels deep you should probably have a think about reorganising your logic.

If Else

Sometimes we want to perform a certain set of actions if a statement is true, and another set of actions if it is false. We can accommodate this with the **else** mechanism.

```

if [ <some test> ]
then
    <commands>
else
    <other commands>
fi

```

Now we could easily read from a file if it is supplied as a command line argument, **else** read from STDIN.

```

else.sh
1.  #!/bin/bash
2.  # else example
3.
4.  if [ $# -eq 1 ]
5.  then
6.      nl $1
7.  else
8.      nl /dev/stdin
9.  fi

```

If Elif Else

Sometimes we may have a series of conditions that may lead to different paths.

```

if [ <some test> ]
then
    <commands>
elif [ <some test> ]
then
    <different commands>
else
    <other commands>
fi

```

For example it may be the case that if you are 18 or over you may go to the party. If you aren't but you have a letter from your parents

you may go but must be back before midnight. Otherwise you cannot go.

```
if_elif.sh
1.  #!/bin/bash
2.  # elif statements
3.
4.  if [ $1 -ge 18 ]
5.  then
6.      echo You may go to the party.
7.  elif [ $2 == 'yes' ]
8.  then
9.      echo You may go to the party but be back before midnight.
10. else
11.     echo You may not go to the party.
12. fi
```

You can have as many elif branches as you like. The final else is also optional.

Boolean Operations

Sometimes we only want to do something if multiple conditions are met. Other times we would like to perform the action if one of several condition is met. We can accommodate these with **boolean operators**.

- **and** - &&
- **or** - ||

For instance maybe we only want to perform an operation if the file is readable **and** has a size greater than zero.

```
and.sh
1.  #!/bin/bash
2.  # and example
3.
4.  if [ -r $1 ] && [ -s $1 ]
5.  then
6.      echo This file is useful.
7.  fi
```

Maybe we would like to perform something slightly different if the user is either bob or andy.

```
or.sh
1.  #!/bin/bash
2.  # or example
3.
4.  if [ $USER == 'bob' ] || [ $USER == 'andy' ]
5.  then
6.      ls -alh
7.  else
8.      ls
9.  fi
```

Case Statements

case statements

Sometimes we may wish to take different paths based upon a variable matching a series of patterns. We could use a series of **if** and **elif** statements but that would soon grow to be unwieldy. Fortunately there is a **case** statement which can make things cleaner. It's a little hard to explain so here are some examples to illustrate:

```
case <variable> in
<pattern 1>
    <commands>
;;
<pattern 2>
    <other commands>
;;
esac
```

Here is a basic example:

```
case.sh
1.  #!/bin/bash
2.  # case example
3.
4.  case $1 in
5.      start)
6.          echo starting
7.          ;;
8.      stop)
9.          echo stoping
10.         ;;
```

```

11.     restart)
12.         echo restarting
13.     ;;
14.     *)
15.         echo don\'t know
16.     ;;
17. esac

```

Let's break it down:

- **Line 4** - This line begins the **case** mechanism.
- **Line 5** - If \$1 is equal to 'start' then perform the subsequent actions. the **)** signifies the end of the pattern.
- **Line 7** - We identify the end of this set of statements with a double semi-colon (**::**). Following this is the next case to consider.
- **Line 14** - Remember that the test for each case is a pattern. The ***** represents any number of any character. It is essentially a catch all if for if none of the other cases match. It is not necessary but is often used.
- **Line 17** - **esac** is case backwards and indicates we are at the end of the case statement. Any other statements after this will be executed normally.

Terminal	
1.	user@bash: ./case.sh start
2.	starting
3.	user@bash: ./case.sh restart
4.	restarting
5.	user@bash: ./case.sh blah
6.	don't know
7.	user@bash:

Now let's look at a slightly more complex example where patterns are used a bit more.

disk_usage.sh	
1.	#!/bin/bash
2.	<i># Print a message about disk usage.</i>
3.	
4.	space_free=\$(df -h awk '{ print \$5 }' sort -n tail -n 1 sed 's/%/')
5.	
6.	case \$space_free in
7.	[1-5]*)
8.	echo Plenty of disk space available
9.	;;
10.	[6-7]*)
11.	echo There could be a problem in the near future
12.	;;
13.	8*)
14.	echo Maybe we should look at clearing out old files
15.	;;
16.	9*)
17.	echo We could have a serious problem on our hands soon
18.	;;
19.	*)
20.	echo Something is not quite right here
21.	;;
22.	esac

Summary

Stuff We Learnt

- if**
Perform a set of commands if a test is true.
- else**
If the test is not true then perform a different set of commands.
- elif**
If the previous test returned false then try this one.
- &&**
Perform the and operation.
- ||**
Perform the or operation.
- case**
Choose a set of commands to execute depending on a string matching a particular pattern.

Important Concepts

- Indenting**
Indenting makes your code much easier to read. It gets increasingly important as your Bash scripts get longer.
- Planning**
Now that your scripts are getting a little more complex you will probably want to spend a little bit of time thinking about how you structure them before diving in.

Activities

Now let's make some decisions.

- Create a Bash script which will take 2 numbers as command line arguments. It will print to the screen the larger of the two numbers.
- Create a Bash script which will accept a file as a command line argument and analyse it in certain ways. eg. you could check if the file is executable or writable. You should print a certain message if true and another if false.
- Create a Bash script which will print a message based upon which day of the week it is (eg. 'Happy hump day' for Wednesday, 'TGIF' for Friday etc).

[Loops](#) ➤

Loops!

Round and round we go.



SECTION BREAKDOWN

[Introduction](#)
[While Loops](#)
[Until Loops](#)
[For Loops](#)
[Break and Continue](#)
[Select](#)
[Summary](#)
[Activities](#)
[NEXT SECTION](#)
[Functions](#)

Introduction

Bash loops are very useful. In this section of our Bash Scripting Tutorial we'll look at the different loop formats available to us as well as discuss when and why you may want to use each of them.

Loops allow us to take a series of commands and keep re-running them until a particular situation is reached. They are useful for automating repetitive tasks.

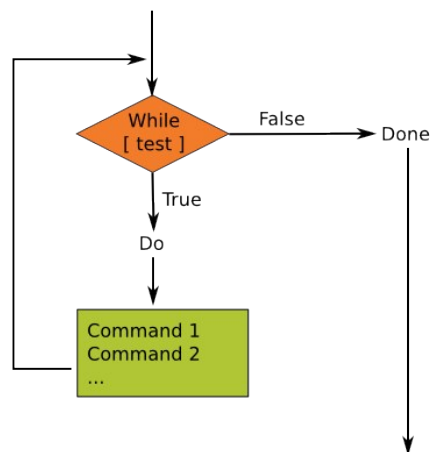
There are 3 basic loop structures in Bash scripting which we'll look at below. There are also a few statements which we can use to control the loops operation.

While Loops

One of the easiest loops to work with is **while** loops. They say, while an expression is true, keep executing these lines of code. They have the following format:

```
while [ <some test> ]
do
  <commands>
done
```

You'll notice that similar to [if statements](#) the test is placed between square brackets [].



In the example below we will print the numbers 1 through to 10:

```

while_loop.sh
1.  #!/bin/bash
2.  # Basic while Loop
3.
4.  counter=1
5.  while [ $counter -le 10 ]
6.  do
7.      echo $counter
8.      ((counter++))
9.  done
10.
11.  echo All done
```

Let's break it down:

- **Line 4** - We'll initialise the variable **counter** with it's starting value.
- **Line 5** - While the test is true (counter is less than or equal to 10) let's do the following commands.

- **Line 7** - We can place any commands here we like. Here **echo** is being used as it's an easy way to illustrate what is going on.
- **Line 8** - Using the double brackets we can increase the value of **counter** by 1.
- **Line 9** - We're at the bottom of the loop so go back to line 5 and perform the test again. If the test is true then execute the commands. If the test is false then continue executing any commands following **done**.

```

Terminal
1. user@bash: ./while_loop.sh
2. 1
3. 2
4. 3
5. 4
6. 5
7. 6
8. 7
9. 8
10. 9
11. 10
12. All done
13. user@bash:

```

Tip

A common mistake is what's called an **off by one** error. In the example above we could have put **-lt** as opposed to **-le** (less than as opposed to less than or equal). Had we done this it would have printed up until 9. These mistakes are easy to make but also easy to fix once you've identified it so don't worry too much if you make this error.

Until Loops

The **until** loop is fairly similar to the while loop. The difference is that it will execute the commands within it until the test becomes true.

```

until [ <some test> ]
do
    <commands>
done

```

```

until_loop.sh
1. #!/bin/bash
2. # Basic until Loop
3.
4. counter=1
5. until [ $counter -gt 10 ]
6. do
7.     echo $counter
8.     ((counter++))
9. done
10.
11. echo All done

```

As you can see in the example above, the syntax is almost exactly the same as the while loop (just replace while with until). We can also create a script that does exactly the same as the while example above just by changing the test accordingly.

So you may be asking, 'Why bother having the two different kinds of loops?'. We don't necessarily. The while loop would be able to handle every scenario. Sometimes, however, it just makes it a little easier to read if we phrase it with **until** rather than **while**. Think about the following statement:

Leave the towel on the line until it's dry.

We could have said:

Leave the towel on the line while it is not dry.

Or:

Leave the towel on the line while it is wet.

But they just don't seem as elegant and easy to understand. So by having both **while** and **until** we can pick whichever one makes the most sense to us and as a result, end up with code that is easier for us to understand when we read it.

Tip

We should always strive for **clean**, **obvious** and **elegant** code when writing our Bash scripts.

For Loops

The **for** loop is a little bit different to the previous two loops. What it does is say for each of the items in a given list, perform the given set of commands. It has the following syntax.

```
for var in <list>
do
    <commands>
done
```

The for loop will take each item in the list (in order, one after the other), assign that item as the value of the variable **var**, execute the commands between do and done then go back to the top, grab the next item in the list and repeat over.

The list is defined as a series of strings, separated by spaces.

Here is a simple example to illustrate:

```
for_loop.sh
1.  #!/bin/bash
2.  # Basic for Loop
3.
4.  names='Stan Kyle Cartman'
5.
6.  for name in $names
7.  do
8.      echo $name
9.  done
10.
11.  echo All done
```

Let's break it down:

- **Line 4** - Create a simple list which is a series of names.
- **Line 6** - For each of the items in the list **\$names** assign the item to the variable **\$name** and do the following commands.
- **Line 8** - echo the name to the screen just to show that the mechanism works. We can have as many commands here as we like.
- **Line 11** - echo another command to show that the bash script continued execution as normal after all the items in the list were processed.

```
Terminal
1.  user@bash: ./for_loop.sh
2.  Stan
3.  Kyle
4.  Cartman
5.  All done
6.  user@bash:
```

Ranges

We can also process a series of numbers

```
for_loop_series.sh
1.  #!/bin/bash
2.  # Basic range in for Loop
3.
4.  for value in {1..5}
5.  do
6.      echo $value
7.  done
8.
9.  echo All done
```

- **Line 4** - It's important when specifying a range like this that there are no spaces present between the curly brackets { }. If there are then it will not be seen as a range but as a list of items.

```
Terminal
1.  user@bash: ./for_loop_series.sh
2.  1
3.  2
4.  3
5.  4
6.  5
7.  All done
8.  user@bash:
```

When specifying a range you may specify any number you like for both the starting value and ending value. The first value may also be larger than the second in which case it will count down.

It is also possible to specify a value to increase or decrease by each time. You do this by adding another two dots (..) and the value to step by.

```
for_loop_stepping.sh
```

```

1.  #!/bin/bash
2.  # Basic range with steps for Loop
3.
4.  for value in {10..0..2}
5.  do
6.      echo $value
7.  done
8.
9.  echo All done

```

Terminal

```

1.  user@bash: ./for_loop.sh
2.  10
3.  8
4.  6
5.  4
6.  2
7.  0
8.  All done
9.  user@bash:

```

One of the more useful applications of **for** loops is in the processing of a set of files. To do this we may use [wildcards](#). Let's say we want to convert a series of .html files over to .php files.

convert_html_to_php.sh

```

1.  #!/bin/bash
2.  # Make a php copy of any html files
3.
4.  for value in $1/*.html
5.  do
6.      cp $value $1/${ basename -s .html $value }.php
7.  done

```

Controlling Loops: Break and Continue

Most of the time your loops are going to through in a smooth and orderly manner. Sometimes however we may need to intervene and alter their running slightly. There are two statements we may issue to do this.

Break

The **break** statement tells Bash to leave the loop straight away. It may be that there is a normal situation that should cause the loop to end but there are also exceptional situations in which it should end as well. For instance, maybe we are copying files but if the free disk space get's below a certain level we should stop copying.

copy_files.sh

```

1.  #!/bin/bash
2.  # Make a backup set of files
3.
4.  for value in $1/*
5.  do
6.      used=$( df $1 | tail -1 | awk '{ print $5 }' | sed 's/%/' )
7.      if [ $used -gt 90 ]
8.      then
9.          echo Low disk space 1>&2
10.         break
11.      fi
12.      cp $value $1/backup/
13.  done

```

Continue

The **continue** statement tells Bash to stop running through this iteration of the loop and begin the next iteration. Sometimes there are circumstances that stop us from going any further. For instance, maybe we are using the loop to process a series of files but if we happen upon a file which we don't have the read permission for we should not try to process it.

copy_check.sh

```

1.  #!/bin/bash
2.  # Make a backup set of files
3.
4.  for value in $1/*
5.  do
6.      if [ ! -r $value ]
7.      then
8.          echo $value not readable 1>&2
9.          continue
10.     fi
11.     cp $value $1/backup/
12.  done

```

Select

The **select** mechanism allows you to create a simple menu system. It has the following format:

```
select var in <list>
do
  <commands>
done
```

When invoked it will take all the items in **list** (similar to other loops this is a space separated set of items) and present them on the screen with a number before each item. A prompt will be printed after this allowing the user to select a number. When they select a number and hit *enter* the corresponding item will be assigned to the variable **var** and the commands between do and done are run. Once finished a prompt will be displayed again so the user may select another option.

A few points to note:

- No error checking is done. If the user enters something other than a number or a number not corresponding to an item then **var** becomes null (empty)
- If the user hits *enter* without entering any data then the list of options will be displayed again.
- The loop will end when an EOF signal is entered or the break statement is issued.
- You may change the system variable **PS3** to change the prompt that is displayed.

Here is a simple example to illustrate it's usage:

```
select_example.sh
1.  #!/bin/bash
2.  # A simple menu system
3.
4.  names='Kyle Cartman Stan Quit'
5.
6.  PS3='Select character: '
7.
8.  select name in $names
9.  do
10.     if [ $name == 'Quit' ]
11.     then
12.         break
13.     fi
14.     echo Hello $name
15. done
16.
17. echo Bye
```

Let's break it down:

- **Line 4** - Set up a variable with the list of characters and a last option which we may select to quit. Note that the items are separated by a space.
- **Line 6** - Change the value of the system variable **PS3** so that the prompt is set to something a little more descriptive. (By default it is #?)
- **Lines 10 - 13** - If the last option, 'Quit', is selected then **break** out of the **select** loop.
- **Line 14** - Print out a message just to demonstrate the mechanism has worked. You may have as many commands here as you like.
- **Line 17** - Print a message just to show that the script has continued as normal after the select loop.

And now let's run the Bash script:

```
Terminal
1.  user@bash: ./select_example.sh
2.  1) Kyle      3) Stan
3.  2) Cartman  4) Quit
4.  Select character: 2
5.  Hello Cartman
6.  Select Character: 1
7.  Hello Kyle
8.  Select character: 4
9.  Bye
10. user@bash:
```

Summary

Stuff We Learnt

while do done

Perform a set of commands while a test is true.

until do done

Perform a set of commands until a test is true.

for do done

Perform a set of commands for each item in a list.

break

Exit the currently running loop.

continue

Stop this iteration of the loop and begin the next iteration.

select do done

Display a simple menu system for selecting items from a list.

Important Concepts

Clarity

There are several Bash loop mechanisms. Pick the one which makes your code the easiest to follow.

Planning

Now that your scripts are getting a little more complex you will probably want to spend a little bit of time thinking about how you structure them before diving in.

Activities

Now we have quite a collection of tools under our belt, we can tackle some more interesting problems.

- Create a simple script which will print the numbers 1 - 10 (each on a separate line) and whether they are even or odd.
- Write a Bash script which will take a single command line argument (a directory) and will print each entry in that directory. If the entry is a file it will print it's size. If the entry is a directory it will print how many items are in that directory.
- Create a command line version of the game [Mastermind](#). Instead of coloured marbles you could use letters or numbers or be creative and find another way.
- Create a command line version of the game Tic Tac Toe. Make it so you can play against the computer.

[Functions](#) ➤

Functions!

Divide and Conquer.



SECTION BREAKDOWN

- [Introduction](#)
- [Functions](#)
- [Variable Scope](#)
- [Overriding Commands](#)
- [Design](#)
- [Summary](#)
- [Activities](#)
- [NEXT SECTION](#)
- [User Interface](#)

Introduction

Functions in Bash Scripting are a great way to reuse code. In this section of our Bash scripting tutorial you'll learn how they work and what you can do with them.

Think of a function as a small script within a script. It's a small chunk of code which you may call multiple times within your script. They are particularly useful if you have certain tasks which need to be performed several times. Instead of writing out the same code over and over you may write it once in a function then call that function every time.

Functions

Creating a function is fairly easy. They may be written in two different formats:

```
function_name () {
    <commands>
}
```

or

```
function function_name {
    <commands>
}
```

A few points to note:

- Either of the above methods of specifying a function is valid. Both operate the same and there is no advantage or disadvantage to one over the other. It's really just personal preference.
- In other programming languages it is common to have arguments passed to the function listed inside the brackets (). In Bash they are there only for decoration and you never put anything inside them.
- The function definition (the actual function itself) must appear in the script before any calls to the function.

Let's look at a simple example:

function_example.sh

```
1.  #!/bin/bash
2.  # Basic function
3.
4.  print_something () {
5.      echo Hello I am a function
6.  }
7.
8.  print_something
9.  print_something
```

Let's break it down:

- **Line 4** - We start defining the function by giving it a name.
- **Line 5** - Within the curly brackets we may have as many commands as we like.
- **Lines 8 and 9** - Once the function has been defined, we may call it as many times as we like and it will execute those commands.

Terminal

```
1.  user@bash: ./function_example.sh
2.  Hello I am a function
3.  Hello I am a function
4.  user@bash:
```

Best Practice

Passing Arguments

It is often the case that we would like the function to process some data for us. We may send data to the function in a similar way to passing command line arguments to a script. We supply the arguments directly after the function name. Within the function they are accessible as **\$1**, **\$2**, etc.

```
arguments_example.sh
1.  #!/bin/bash
2.  # Passing arguments to a function
3.
4.  print_something () {
5.      echo Hello $1
6.  }
7.
8.  print_something Mars
9.  print_something Jupiter
```

```
Terminal
1.  user@bash: ./arguments_example.sh
2.  Hello Mars
3.  Hello Jupiter
4.  user@bash:
```

Return Values

Most other programming languages have the concept of a return value for functions, a means for the function to send data back to the original calling location. Bash functions don't allow us to do this. They do however allow us to set a return status. Similar to how a program or command exits with an exit status which indicates whether it succeeded or not. We use the keyword **return** to indicate a return status.

```
return_status_example.sh
1.  #!/bin/bash
2.  # Setting a return status for a function
3.
4.  print_something () {
5.      echo Hello $1
6.      return 5
7.  }
8.
9.  print_something Mars
10. print_something Jupiter
11. echo The previous function has a return value of $?
```

Let's break it down

- **Line 6** - The return status doesn't have to be hardcoded. It may be a variable
- **Line 11** - Remember that the variable **\$?** contains the return status of the previously run command or function.

```
Terminal
1.  user@bash: ./return_status_example.sh
2.  Hello Mars
3.  Hello Jupiter
4.  The previous function has a return value of 5
5.  user@bash:
```

Tip

Typically a return status of 0 indicates that everything went successfully. A non zero value indicates an error occurred.

If all you want to do is return a number (eg. the result of a calculation) then you can consider using the return status to achieve this. It is not its intended purpose but it will work.

One way to get around this is to use [Command Substitution](#) and have the function print the result (and only the result).

```
return_hack.sh
1.  #!/bin/bash
2.  # Setting a return value to a function
3.
4.  lines_in_file () {
5.      cat $1 | wc -l
6.  }
7.
8.  num_lines=$( lines_in_file $1 )
9.
10. echo The file $1 has $num_lines lines in it.
```

Let's break it down:

- **Line 5** - This command will print the number of lines in the file referred to by \$1.
- **Line 8** - We use command substitution to take what would normally be printed to the screen and assign it to the variable `num_lines`

```
Terminal
1. user@bash: cat myfile.txt
2. Tomato
3. Lettuce
4. Capsicum
5. user@bash: ./return_hack.sh myfile.txt
6. The file myfile.txt has 3 lines in it.
7. user@bash:
```

Just be wary if you take this approach as if you don't call the function with command substitution then it will print the result to the screen. Sometimes that is ok because that is what you want. Other times that may be undesirable.

Variable Scope

Scope refers to which parts of a script can see which variables. By default a variable is **global**. This means that it is visible everywhere in the script. We may also create a variable as a **local** variable. When we create a local variable within a function, it is only visible within that function. To do that we use the keyword **local** in front of the variable the first time we set it's value.

```
local var_name=<var_value>
```

It is generally considered good practice to use local variables within functions so as to keep everything within the function contained. This way variables are safer from being inadvertently modified by another part of the script which happens to have a variable with the same name (or vice versa).

```
local_variables.sh
1. #!/bin/bash
2. # Experimenting with variable scope
3.
4. var_change () {
5.     local var1='local 1'
6.     echo Inside function: var1 is $var1 : var2 is $var2
7.     var1='changed again'
8.     var2='2 changed again'
9. }
10.
11. var1='global 1'
12. var2='global 2'
13.
14. echo Before function call: var1 is $var1 : var2 is $var2
15.
16. var_change
17.
18. echo After function call: var1 is $var1 : var2 is $var2
```

```
Terminal
1. user@bash: ./local_variables.sh
2. Before function call: var1 is global 1 : var2 is global 2
3. Inside function: var1 is local 1 : var2 is global 2
4. After function call: var1 is global 1 : var2 is 2 changed again
5. user@bash:
```

Best Practice

Always use local variables within functions. Use global variables as a last resort and consider if there is a better way to do it before using them.

Scope can sometimes be hard to get your head around at first. If it seems a bit confusing, the best approach is to create a Bash script similar to the one above and tweak it several times setting and changing variables in different places then observing the behaviour when you run it.

Overriding Commands

It is possible to name a function as the same name as a command you would normally use on the command line. This allows us to create a wrapper. eg. Maybe every time we call the command **ls** in our script, what we actually want is **ls -lh**. We could do the following:

```
override.sh
```



```

1.  #!/bin/bash
2.  # Create a wrapper around the command ls
3.
4.  ls () {
5.      command ls -lh
6.  }
7.
8.  ls

```

Let's break it down:

- **Line 5** - When we have a function with the same name as a command we need to put the keyword **command** in front of the the name when we want the command as opposed to the function as the function normally takes precedence.

In the example above, if we didn't put the keyword **command** in front of `ls` on line 5 we would end up in an endless loop. Even though we are inside the function `ls` when we call `ls` it would have called another instance of the function `ls` which in turn would have done the same and so on.

Tip

It's easy to forget the **command** keyword and end up in an endless loop. If you encounter this then you can cancel the script from running by pressing the keys **CTRL c** at the same time on your keyboard. **CTRL c** is a good way to cancel your script (or a program) whenever you get into trouble on the command line.

Design

Creating functions in your Bash scripts is easy. Creating good functions that make your scripts easier to write and maintain takes time and experience however. As with most things with computers when you get to this level of complexity, there will be several ways you could achieve the desired outcome. Some will be better than others so take the time to think about different ways you could write your code and which way may be better.

Sometimes better is least lines of code, sometimes better is easiest to modify later if requirements change. Sometimes better is the approach which is least prone to errors.

If a particular task needs to be performed several times then it is a good candidate for placing within a function.

Sometimes it is good to put ancillary tasks within functions too so that they are logically separate from the main part of the script. A common example is validating input (eg. making sure a specified file exists and is readable).

A function is most reuseable when it performs a single task and a single task only. Instead of having a large function, consider breaking it up into several functions and breaking the task up.

You need to find the right balance however. If the functions are too large and take on too much processing then you don't get the full benefit. If you divide up into too many functions then your code can easily grow and become silly. With experience you will find that sweet spot in the middle.

Summary

Stuff We Learnt

function <name> or <name> ()

Create a function called **name**.

return <value>

Exit the function with a return status of value.

local <name>=<value>

Create a local variable within a function.

command <command>

Run the command with that name as opposed to the function with the same name.

Important Concepts

Re-use

Functions allow us to easily re-use code making the code easier to manage and read.

Planning

Now that your scripts are getting a little more complex you will probably want to spend a little bit of time thinking about how you structure them before diving in.

Activities

For this section there aren't any activities. What I suggest you do is go back to the [activities from the previous section](#) and redo them using functions.

User Interface!

Beauty is more than skin deep.



SECTION BREAKDOWN

[Introduction](#)

[TPut](#)

[Supplying Data](#)

[Presenting Data](#)

[Organising Your Code](#)

[Final Word](#)

Introduction

This is the final section in the tutorial and I'd like to use it to discuss a very important topic (which is often neglected) the user interface. I've touched on various points regarding the user interface throughout the tutorial but here I'll bring them all together and introduce a few other concepts as well.

When most people think about the user interface they think about the bits the end user sees and how they interact with the tool. For Bash scripts I like to think about the layout and structure of the commands inside the script as well. Bash scripts are often small tools used to automate tedious and repetitive tasks. They are always readable by the end user and often modified to suit changing requirements. Therefore the ease with which the user (often yourself) may modify and extend the script is also very important.

TPut

tput is a command which allows you to control the cursor on the terminal and the format of content that is printed. It is quite a powerful and complex tool so I'll introduce some of the basics here but leave it up to you to do further research.

Here is an example printing a message in the center of the screen.

```
center_message.sh

1.  #!/bin/bash
2.  # Print message in center of terminal
3.
4.  cols=$( tput cols )
5.  rows=$( tput lines )
6.
7.  message=$@
8.
9.  input_length=${#message}
10.
11. half_input_length=$(( $input_length / 2 ))
12.
13. middle_row=$(( $rows / 2 ))
14. middle_col=$(( ($cols / 2) - $half_input_length ))
15.
16. tput clear
17.
18. tput cup $middle_row $middle_col
19. tput bold
20. echo $@
21. tput sgr0
22. tput cup $( tput lines ) 0
```

Let's break it down:

- **Line 4** - **tput cols** will tell us how many columns the terminal has.
- **Line 5** - **tput lines** will tell us how many lines (or rows) the terminal has.
- **Line 7** - Take all the command line arguments and assign them to a single variable **message**.
- **Line 9** - Find out how many characters are in the string **message**. We had to assign all the input values to the variable **message** first as **\${#@}** would tell us how many command line arguments there were instead of the number of characters combined.
- **Line 11** - We need to know what 1/2 the length of the string **message** is in order to center it.
- **Lines 13 and 14** - Calculate where to place the message for it to be centered.
- **Line 16** - **tput clear** will clear the terminal.
- **Line 18** - **tput cup** will place the cursor at the given row and column.
- **Line 19** - **tput bold** will make everything printed to the screen bold.
- **Line 20** - Now we have everything set up let's print our message.
- **Line 21** - **tput sgr0** will turn bold off (and any other changes we may have made).
- **Line 22** - Place the prompt at the bottom of the screen.

```
Terminal
1. user@bash: ./center_message.sh Hello there
2.
3.
4.
5.
6.
7. user@bash:
Hello there
```

Note: Normally the first prompt (where we run the script) would be removed with the clear command. We have left it here only so you can see that it was run to get the script started.

With **tput** and a bit of creativity you can create some really interesting effects. Especially so if you delay actions using the command **sleep**. Only use it when appropriate however. Most of the time just printing the processed data (without formatting) is more convenient for the user.

Supplying Data

Remember there are 3 ways in which you may supply data to a Bash script:

- As command line arguments
- Redirected in as STDIN
- Read interactively during script execution

Your script may use one or a combination of these but should always aim to be the most convenient for the user.

Command line arguments are good as they will be retained in the users history making it easy for them to rerun commands. Command line arguments are also convenient when the script is not run directly by the user (eg, as part of another script or a cron task etc).

Redirected from STDIN is good when your script is behaving like a filter and just modifying or reformatting data that is fed to it.

Reading interactively is good when you don't know what data may be required until the script is already running. eg. You may need to clarify some suspicious or erroneous input. Passwords are also ideally asked for this way so they aren't kept as plain text in the users history.

Input Flexibility

Think about how strict you are going to be with supplied data as well. The more flexible you can be the happier the end user is going to be. Think of someone supplying a date as an argument. They could supply the date as:

23-12-2021

or

23/12/2021

or

23:12:2021

We could write our script to insist on input in only one particular format. This would be easiest for us but potentially not convenient for the end user. What if they want to feed the date in as provided from another command or source that provides it in a different format?

We should always aim to be most convenient for the end user as opposed to ourselves. After all, we'll write it once but they will run it many times.

The command **sed** can easily allow us to accommodate many formats for input data.

```
flexible_date.sh
1. #!/bin/bash
2. # A date is the first command line argument
3.
4. clean_date=$( echo $1 | sed 's/[ /:\^#]/-/g' )
5.
6. echo $clean_date
```

Presenting Data

Remember that the terminal and the nature of the commands you use there are typically a little different to your normal interaction with computers in a graphical user interface. Again we want what is most convenient for the user. Often this is just to print the output as a plain result, without any formatting or fancy messages surrounding it. Then it is easiest for the user to redirect the output into other commands for further processing or to a file for saving.

Organising Your Code

Presentation of your code is very important and you should take pride in it. Good structure makes it easier for you to see what the code is doing and harder to make silly mistakes (which can easily waste a lot of time or potentially worse if you don't realise the mistake).

It's common to take the approach of *'yeah yeah, that's other people though, I don't make those silly mistakes so I can be lazy and*

write sloppy code and it'll be fine. Everyone can make mistakes, even [NASA](#). Take the time to structure your code well and later on you'll be thankful you did.

- Indent your code and space it out well so that different sections are easily distinguished.
- Name variables and functions with descriptive names so it is clear what they represent and do.
- Use comments where appropriate to explain a bit of code who's operation is not immediately obvious.

Final Word

Ok so you've worked through my Bash tutorial. Congratulations, you've now aquired some very powerful and useful skills and knowledge. Next you need to gain experience. Your Bash scripting foo is no doubt reasonably good now but it will only get better with practice.

Tip

Speaking of practice, why not try some of our [programming challenges](#).

Remember too that this tutorial is not a complete reference on Bash scripting. I've tried to cover the most important and essential bits. For a lot of you this will be more than enough to automate tasks and make your interaction with Linux much happier. For others, you will want to take things further and there is much more to Bash that you can learn. You now have a solid foundation to launch from and should have no troubles extending your abilities. Either way, I hope your experiences with Linux are, and continue to be, **awesome** and I wish you the best of luck.

Also, if you have any feedback on this tutorial (or any of my others) I would be happy to hear from you. It could be a typo you have spotted, or some other error, a bit you feel could be written more clearly, or just that you found it useful. While I can't guarantee to act upon all feedback I do very much appreciate it.

You may drop me a message at ryantutorial@todolistme.net

More importantly (especially if you found this tutorial useful), **don't keep it a secret**. The best way to say thank you is to make sure you share this tutorial on social media (Twitter, Facebook, Google Plus or whatever the hip and happening social platform of today is), give us a shout out on your blog, tell your friends and co-workers, etc.

Cheers

Ryan

✉

Welcome

Welcome to the learnshell.org interactive Shell Programming tutorial.

Whether you are an experienced programmer or not, this website is intended for everyone who wishes to learn programming with Unix/Linux shell interpreters.

You are welcome to join our group on Facebook for questions, discussions and updates.

Just click on the chapter you wish to begin from, and follow the instructions. Good luck!

Learn the Basics

- [Hello, World!](#)
- [Variables](#)
- [Passing Arguments to the Script](#)
- [Arrays](#)
- [Basic Operators](#)
- [Basic String Operations](#)
- [Decision Making](#)
- [Loops](#)
- [Array-Comparison](#)
- [Shell Functions](#)

Advanced Tutorials

- [Special Variables](#)
- [Bash trap command](#)
- [File Testing](#)
- [Input Parameter Parsing](#)
- [Pipelines](#)
- [Process Substitution](#)
- [Regular Expressions](#)
- [Special Commands sed,awk,grep,sort](#)

[Welcome](#) / **Hello, World!**

Hello, World!

The tutorial discusses shell programming in general with focus on **Bash** ("Bourne Again Shell") shell as the main shell interpreter. Shell programming using other common shells such as sh, csh, tcsh, will also be referenced, as they sometime differ from bash.

Shell programming can be accomplished by directly executing shell commands at the shell prompt or by storing them in the order of execution, in a text file, called a shell script, and then executing the shell script. To execute, simply write the shell script file name, once the file has execute permission (chmod +x filename).

The first line of the shell script file begins with a "sha-bang" (!) which is not read as a comment, followed by the full path where the shell interpreter is located. This path, tells the operating system that this file is a set of commands to be fed into the interpreter indicated. Note that if the path given at the "sha-bang" is incorrect, then an error message e.g. "Command not found.", may be the result of the script execution. It is common to name the shell script with the ".sh" extension. The first line may look like this:

#!/bin/bash

Adding comments: any text following the "#" is considered a comment

To find out what is currently active shell, and what is its path, type the highlighted command at the shell prompt (sample responses follow):

ps | grep \$\$

```
987 tty1 00:00:00 bash
```

This response shows that the shell you are using is of type 'bash'. next find out the full path of the shell interpreter

which bash

```
/bin/bash
```

This response shows the full execution path of the shell interpreter. Make sure that the "sha-bang" line at the beginning of your script, matches this same execution path.

Exercise

Use the "echo" command to print the line "Hello, World!".

Variables

Shell variables are created once they are assigned a value. A variable can contain a number, a character or a string of characters. Variable name is case sensitive and can consist of a combination of letters and the underscore "_". Value assignment is done using the "=" sign. Note that no space permitted on either side of = sign when initializing variables.

```
PRICE_PER_APPLE=5
MyFirstLetters=ABC
greeting='Hello      world!'
```

Referencing the variables

A backslash "\" is used to escape special character meaning

```
PRICE_PER_APPLE=5
echo "The price of an Apple today is: \$HK $PRICE_PER_APPLE"
```

Encapsulating the variable name with \${} is used to avoid ambiguity

```
MyFirstLetters=ABC
echo "The first 10 letters in the alphabet are: ${MyFirstLetters}DEFGHIJ"
```

Encapsulating the variable name with "" will preserve any white space values

```
greeting='Hello      world!'
echo "$greeting" now with spaces: $greeting"
```

Variables can be assigned with the value of a command output. This is referred to as substitution. Substitution can be done by encapsulating the command with `` (known as back-ticks) or with \${}

```
FILELIST=`ls`
FileWithTimeStamp=/tmp/my-dir/file_${/bin/date +%Y-%m-%d}.txt
```

Note that when the script runs, it will run the command inside the \${} parenthesis and capture its output.

Exercise

The target of this exercise is to create a string, an integer, and a complex variable using command substitution. The string should be named BIRTHDATE and should contain the text "Jan 1, 2000". The integer should be named Presents and should contain the number 10. The complex variable should be named BIRTHDAY and should contain the full weekday name of the day matching the date in variable BIRTHDATE e.g. Saturday. Note that the 'date' command can be used to convert a date format into a different date format. For example, to convert date value, \$date1, to day of the week of date1, use:

```
date -d "$date1" +%A
```

Tutorial **Passing Arguments To the Script**

Arguments can be passed to the script when it is executed, by writing them as a space-delimited list following the script file name.

Inside the script, the \$1 variable references the first argument in the command line, \$2 the second argument and so forth. The variable \$0 references to the current script. In the following example, the script name is followed by 6 arguments.

```
./bin/my_shopping.sh apple 5 banana 8 "Fruit Basket" 15
```

```
echo $3 --> results with: banana
```

```
BIG=$5
```

```
echo "A $BIG costs just $6" --> results with: A Fruit Basket costs just 15
```

The variable \$# holds the number of arguments passed to the script

```
echo $# --> results with: 6
```

The variable \$@ holds a space delimited string of all arguments passed to the script

Exercise

Pass "Shell is fun" (3 arguments) to the script as an array and print the length of the array.

Tutorial Code

```
#!/bin/bash
#To pass three arguments
args=("$@")
echo ${args[0]} ${args[1]} ${args[2]}
echo $#                                #prints the length f the array
```

Solution

```
#!/bin/bash args=("Shell" "is" "fun") echo ${args[0]} ${args[1]} ${args[2]} echo ${#args[@]}
```

Expected Output

```
Shell is fun
3
```

Arrays

An array can hold several values under one name. Array naming is the same as variables naming. An array is initialized by assign space-delimited values enclosed in ()

```
my_array=(apple banana "Fruit Basket" orange)
new_array[2]=apricot
```

Array members need not be consecutive or contiguous. Some members of the array can be left uninitialized.

The total number of elements in the array is referenced by `${#arrayname[@]}`

```
my_array=(apple banana "Fruit Basket" orange)
echo ${#my_array[@]}           # 4
```

The array elements can be accessed with their numeric index. The index of the first element is 0.

```
my_array=(apple banana "Fruit Basket" orange)
echo ${my_array[3]}           # orange - note that curly brackets are needed
# adding another array element
my_array[4]="carrot"           # value assignment without a $ and curly brackets
echo ${my_array[@]}           # 5
echo ${my_array[${#my_array[@]}-1]} # carrot
```

Exercise

In this exercise, you will need to add numbers and strings to the correct arrays. You must add the numbers 1,2, and 3 to the "numbers" array, and the words 'hello' and 'world' to the strings array.

You will also have to correct the values of the variable NumberOfNames and the variable second_name. NumberOfNames should hold the total number of names in the NAMES array, using the `$#` special variable. Variable second_name should hold the second name in the NAMES array, using the brackets operator `[ ]`. Note that the index is zero-based, so if you want to access the second item in the list, its index will be 1.

[Welcome](#) / **Basic Operators**

Basic Operators

Arithmetic Operators

Simple arithmetics on variables can be done using the arithmetic expression: `$((expression))`

`A=3`

`B=$((100 * $A + 5)) # 305`

The basic operators are:

a + b addition (a plus b)

a - b substraction (a minus b)

a * b multiplication (a times b)

a / b division (integer) (a divided by b)

a % b modulo (the integer remainder of a divided by b)

a ** b exponentiation (a to the power of b)

Exercise

In this exercise, you will need to calculate to total cost (variable TOTAL) of a fruit basket, which contains 1 pineapple, 2 bananas and 3

Basic String Operations

The shell allows some common string operations which can be very useful for script writing.

String Length

```
#          1234567890123456
STRING="this is a string"
echo ${#STRING}           # 16
```

Index

Find the numerical position in \$STRING of any single character in \$SUBSTRING that matches. Note that the 'expr' command is used in this case.

```
STRING="this is a string"
SUBSTRING="hat"
expr index "$STRING" "$SUBSTRING"    # 1 is the position of the first 't' in $STRING
```

Substring Extraction

Extract substring of length \$LEN from \$STRING starting after position \$POS. Note that first position is 0.

```
STRING="this is a string"
POS=1
LEN=3
echo ${STRING:$POS:$LEN}    # his
```

If :\$LEN is omitted, extract substring from \$POS to end of line

```
STRING="this is a string"
echo ${STRING:1}           # $STRING contents without leading character
echo ${STRING:12}          # ring
```

Simple data extraction example:

```
# Code to extract the First name from the data record
DATARECORD="last=Clifford,first=Johnny Boy,state=CA"
COMMA1=`expr index "$DATARECORD" ','` # 14 position of first comma
CHOP1FIELD=${DATARECORD:$COMMA1}      #
COMMA2=`expr index "$CHOP1FIELD" ','`
LENGTH=`expr $COMMA2 - 6 - 1`
FIRSTNAME=${CHOP1FIELD:6:$LENGTH}     # Johnny Boy
echo $FIRSTNAME
```

Substring Replacement

```
STRING="to be or not to be"
```

Replace first occurrence of substring with replacement

```
STRING="to be or not to be"
echo ${STRING[@]/be/eat}      # to eat or not to be
```

Replace all occurrences of substring

```
STRING="to be or not to be"
echo ${STRING[@]//be/eat}     # to eat or not to eat
```

Delete all occurrences of substring (replace with empty string)

```
STRING="to be or not to be"
echo ${STRING[@]// not/}      # to be or to be
```

Replace occurrence of substring if at the beginning of \$STRING

```
STRING="to be or not to be"
echo ${STRING[@]/#to be/eat now} # eat now or not to be
```

Replace occurrence of substring if at the end of \$STRING

```
STRING="to be or not to be"
echo ${STRING[@]/%be/eat}       # to be or not to eat
```

replace occurrence of substring with shell command output

```
STRING="to be or not to be"
echo ${STRING[@]/%be/be on $(date +%Y-%m-%d)} # to be or not to be on 2012-06-14
```

Exercise

In this exercise, you will need to change Warren Buffett's known saying. First create a variable ISAY and assign it the original saying value. Then re-assign it with a new changed value using the string operations and following the 4 defined changes: Change1: replace the first occurrence of 'snow' with 'foot'. Change2: delete the second occurrence of 'snow'. Change3: replace 'finding' with 'getting'. Changed: delete all characters following 'wet'. Tip: One way to implement Change4, if to find the index of 'w' in the word 'wet' and then use substring extraction.

Decision Making

As in popular programming languages, the shell also supports logical decision making.

The basic conditional decision making construct is:

if [expression]; then

code if 'expression' is true

fi

```
NAME="John"
if [ "$NAME" = "John" ]; then
    echo "True - my name is indeed John"
fi
```

It can be expanded with 'else'

```
NAME="Bill"
if [ "$NAME" = "John" ]; then
    echo "True - my name is indeed John"
else
    echo "False"
    echo "You must mistaken me for $NAME"
fi
```

It can be expanded with 'elif' (else-if)

```
NAME="George"
if [ "$NAME" = "John" ]; then
    echo "John Lennon"
elif [ "$NAME" = "George" ]; then
    echo "George Harrison"
else
    echo "This leaves us with Paul and Ringo"
fi
```

The expression used by the conditional construct is evaluated to either true or false. The expression can be a single string or variable. A empty string or a string consisting of spaces or an undefined variable name, are evaluated as false. The expression can be a logical combination of comparisons: negation is denoted by `!`, logical AND (conjunction) is denoted by `&&`, and logical OR (disjunction) is denoted by `||`. Conditional expressions should be surrounded by double brackets `[[]]`.

Types of numeric comparisons

comparison	Evaluated to true when
<code>\$a -lt \$b</code>	<code>\$a < \$b</code>
<code>\$a -gt \$b</code>	<code>\$a > \$b</code>
<code>\$a -le \$b</code>	<code>\$a <= \$b</code>
<code>\$a -ge \$b</code>	<code>\$a >= \$b</code>
<code>\$a -eq \$b</code>	<code>\$a</code> is equal to <code>\$b</code>
<code>\$a -ne \$b</code>	<code>\$a</code> is not equal to <code>\$b</code>

Types of string comparisons

comparison	Evaluated to true when
<code>"\$a" = "\$b"</code>	<code>\$a</code> is the same as <code>\$b</code>
<code>"\$a" == "\$b"</code>	<code>\$a</code> is the same as <code>\$b</code>
<code>"\$a" != "\$b"</code>	<code>\$a</code> is different from <code>\$b</code>
<code>-z "\$a"</code>	<code>\$a</code> is empty

- note1: whitespace around `=` is required
- note2: use `"` around string variables to avoid shell expansion of special characters as `*`

Logical combinations

```
if [[ $VAR_A[0] -eq 1 && ($VAR_B = "bee" || $VAR_T = "tee") ]] ; then
    command...
fi
```

case structure

```
case "$variable" in
    "$condition1" )
        command...
    ;;
    "$condition2" )
        command...
    ;;
esac
```

simple case bash structure

```
mycase=1
case $mycase in
    1) echo "You selected bash";;
    2) echo "You selected perl";;
    3) echo "You selected phyton";;
    4) echo "You selected c++";;
    5) exit
esac
```

Exercise

Change the variables in the first section, so that each if statement resolves as True.

Loops

bash for loop

```
# basic construct
for arg in [list]
do
    command(s)...
done
```

For each pass through the loop, arg takes on the value of each successive value in the list. Then the command(s) are executed.

```
# loop on array member
NAMES=(Joe Jenny Sara Tony)
for N in ${NAMES[@]} ; do
    echo "My name is $N"
done

# loop on command output results
for f in $( ls prog.sh /etc/localtime ) ; do
    echo "File is: $f"
done
```

bash while loop

```
# basic construct
while [ condition ]
do
    command(s)...
done
```

The while construct tests for a condition, and if true, executes commands. It keeps looping as long as the condition is true.

```
COUNT=4
while [ $COUNT -gt 0 ]; do
    echo "Value of count is: $COUNT"
    COUNT=$((COUNT - 1))
done
```

bash until loop

```
# basic construct
until [ condition ]
do
    command(s)...
done
```

The until construct tests for a condition, and if false, executes commands. It keeps looping as long as the condition is false (opposite of while construct)

```
COUNT=1
until [ $COUNT -gt 5 ]; do
    echo "Value of count is: $COUNT"
    COUNT=$((COUNT + 1))
done
```

"break" and "continue" statements

break and continue can be used to control the loop execution of for, while and until constructs. continue is used to skip the rest of a particular loop iteration, whereas break is used to skip the entire rest of loop. A few examples:

```
# Prints out 0,1,2,3,4
```

```
COUNT=0
while [ $COUNT -ge 0 ]; do
    echo "Value of COUNT is: $COUNT"
    COUNT=$((COUNT+1))
    if [ $COUNT -ge 5 ] ; then
        break
    fi
done
```

```
# Prints out only odd numbers - 1,3,5,7,9
```

```
COUNT=0
while [ $COUNT -lt 10 ]; do
    COUNT=$((COUNT+1))
    # Check if COUNT is even
    if [ $((COUNT % 2)) = 0 ] ; then
        continue
    fi
    echo $COUNT
done
```

*Exercise

In this exercise, you will need to loop through and print out all even numbers from the numbers list in the same order they are received. Don't print any numbers that come after 237 in the sequence.

Array-Comparison

Comparison of arrays Shell can handle arrays An array is a variable containing multiple values. Any variable may be used as an array. There is no maximum limit to the size of an array, nor any requirement that member variables be indexed or assigned contiguously. Arrays are zero-based: the first element is indexed with the number 0.

```
# basic construct
# array=(value1 value2 ... valueN)
array=(23 45 34 1 2 3)
#To refer to a particular value (e.g. : to refer 3rd value)
echo ${array[2]}
```

```
#To refer to all the array values
echo ${array[@]}
```

```
#To evaluate the number of elements in an array
echo ${#array[@]}
```

Exercise

In this exercise, you will need to compare three list of arrays and write the common elements of all the three arrays:

`a=(3 5 8 10 6),b=(6 5 4 12),c=(14 7 5 7)` result is the common element 5.

Shell Functions

Like other programming languages, the shell may have functions. A function is a subroutine that implements a set of commands and operations. It is useful for repeated tasks.

```
# basic construct
function function_name {
    command...
}
```

Functions are called simply by writing their names. A function call is equivalent to a command. Parameters may be passed to a function, by specifying them after the function name. The first parameter is referred to in the function as \$1, the second as \$2 etc.

```
function function_B {
    echo "Function B."
}
function function_A {
    echo "$1"
}
function adder {
    echo "$(($1 + $2))"
}
```

```
# FUNCTION CALLS
# Pass parameter to function A
function_A "Function A."      # Function A.
function_B                    # Function B.
# Pass two parameters to function adder
adder 12 56                   # 68
```

Exercise

In this exercise, you will need to write a function called ENGLISH_CALC which can process sentences such as:

'3 plus 5', '5 minus 1' or '4 times 6' and print the results as: '3 + 5 = 8', '5 - 1 = 4' or '4 * 6 = 24' respectively.

Tutorial Special Variables

In last tutorial about shell function, you use "\$1" represent the first argument passed to function_A. Moreover, here are some special variables in shell:

- `$0` - The filename of the current script.|
- `$n` - The Nth argument passed to script was invoked or function was called.|
- `$#` - The number of argument passed to script or function.|
- `$@` - All arguments passed to script or function.|
- `$*` - All arguments passed to script or function.|
- `$?` - The exit status of the last command executed.|
- `$$` - The process ID of the current shell. For shell scripts, this is the process ID under which they are executing.|
- `$_` - The process number of the last background command.|

Example:

```
#!/bin/bash
echo "Script Name: $0"
function func {
    for var in $*
    do
        let i=i+1
        echo "The \${i} argument is: ${var}"
    done
    echo "Total count of arguments: $#"
```

`$@` and `$*` have different behavior when they were enclosed in double quotes.

```
#!/bin/bash
function func {
    echo "--- \"\$*\""
```

Exercise

There is no exercise for this section.

Tutorial File Testing

Often you will want to do some file tests on the file system you are running. In this case, shell will provide you with several useful commands to achieve it.

The command looks like the following

- `-<command> [filename]`
- `[filename1] -<command> [filename2]`

We will briefly introduce some common commands you might encounter in your daily life.

Example

use "-e" to test if file exist

```
#!/bin/bash
filename="sample.md"
if [ -e "$filename" ]; then
    echo "$filename exists as a file"
fi
```

use "-d" to test if directory exists

```
#!/bin/bash
directory_name="test_directory"
if [ -d "$directory_name" ]; then
    echo "$directory_name exists as a directory"
fi
```

use "-r" to test if file has read permission for the user running the script/test

```
#!/bin/bash
filename="sample.md"
if [ ! -f "$filename" ]; then
    touch "$filename"
fi
if [ -r "$filename" ]; then
    echo "you are allowed to read $filename"
else
    echo "you are not allowed to read $filename"
fi
```

Exercise

There is no exercise for this section.

Tutorial

Process Substitution

In the previous section we've seen how to chain output of one command to the next one. But what if you want to chain the output of two or more commands to the another one? What if you have a command that takes a file as argument but you would like to process whatever is send to that file?

Process substitution allows a process's input or output to be referred to using a filename. It has two forms: output `<(cmd)`, and input `>(cmd)`.

Example:

Output

Imagine you've two files for which you want to compare the content. Using `diff file1 file2` could generate false positives in the case lines are not ordered. So if you want to compare those files you could create two new files, ordered, and compare those. It would look like:

```
sort file1 > sorted_file1
sort file2 > sorted_file2
diff sorted_file1 sorted_file2
```

With process substitution you can do it in one line:

```
diff <(sort file1) <(sort file2)
```

Input

Imagine you want to store logs of an application into a file and at the same time print it on the console. A very handy command for that is `tee`.

```
echo "Hello, world!" | tee /tmp/hello.txt
```

Now let say you want to have only lower case characters in the file but keep the regular case on the output. You could use process substitution that way:

```
echo "Hello, world!" | tee >(tr '[:upper:]' '[:lower:]' > /tmp/hello.txt)
```

Exercise

There is no exercise for this section.